Reviewer Pack — Fourier Min-Coefficient Inverse Proportionality to CNF Size

Entry 15c4b78718ca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 08:50:22 UTC

Fourier Min-Coefficient Inverse Proportionality to CNF Size

Entry ID: `15c4b78718ca` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 08:50:22 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis on Boolean Functions **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> For every CNF formula Φ with n variables and m clauses, the minimal absolute Fourier coefficient μ(Φ) satisfies μ(Φ) ≥ c * m^{-1/2} for some absolute constant c > 0. This bound is tight up to polylog(m) factors.

Rationale (proposer's reasoning):

> Fourier coefficients capture correlation with parity functions, while CNF size reflects combinatorial complexity. The inverse square-root relationship suggests structural constraints on how functions can be represented with sparse clauses.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `48fb4ed434f7cadb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=241.6s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def generate_primes(n):
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n), random.randint(-n, -1)]
        cnf.append(clause)
    return cnf

def fast_walsh_hadamard_transform(x):
    n = len(x)
    if n == 1:
        return x
    even = fast_walsh_hadamard_transform(x[0::2])
    odd = fast_walsh_hadamard_transform(x[1::2])
    result = [0] * n
    for k in range(n // 2):
        result[k] = even[k] + odd[k]
        result[k + n // 2] = even[k] - odd[k]
    return result
```

```
def compute_fourier_coefficients(cnf, n):
    num_vars = 2 ** n
    zero_vector = [Fraction(0) for _ in range(num_vars)]
    for clause in cnf:
        term = Fraction(1)
        for literal in clause:
            if literal > 0:
                term *= Fraction(1, 2)
            else:
                term *= Fraction(-1, 2)
        zero_vector[abs(literal) - 1] += term
    return fast_walsh_hadamard_transform(zero_vector)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n * 2, min(n * 10, 100))
    cnf = generate_cnf(n, m)

    mu = compute_fourier_coefficients(cnf, n)
    mu_min = min(abs(coeff) for coeff in mu if abs(coeff) > 0)
    c = Fraction(0.1)
    conjecture_holds = mu_min >= c * (m ** -Fraction(1, 2))

    return {
        "metric_name": "Fourier Min-Coefficient",
        "metric_value": float(mu_min),
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"mu_min={mu_min} < {c * (m ** -Fraction(1,
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109325
2	propose	ollama_remote	qwen3:8b	0	0	111892
3	preregistration	ollama_remote	qwen3:8b	0	0	24524
4	novelty	ollama_remote	qwen3:8b	0	0	20672
5	novelty	ollama_remote	qwen3:8b	0	0	14785
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11937
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10627
8	judge	ollama_remote	qwen3:8b	0	0	124773

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 428535 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/15c4b78718ca.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/15c4b78718ca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/15c4b78718ca.tar.gz` (if generated)